TECHNICAL GUIDE · NOVEMBER 2026

The Al-Bayan Platform

مكتبة البيان

How the whole platform works, from the dev stack to the code to the screens. You'll also see the decisions that make it fast, safe and simple to use. Every code excerpt is real code from the repository, with notes explaining it line by line.

Backend NestJS 11 · TypeScript · Prisma 5 · PostgreSQL	**Web apps** React 19 · Vite 8 · Tailwind 4 · TanStack Query	**Mobile** Flutter · two apps, same API
Files Cloudflare R2 · sharp image pipeline	**Hosting** Render (Docker + static) · Neon Postgres	**Quality** 151 unit tests · strict types · 0 known vulnerabilities

github.com/kouteiba97/BookStore

20, rue Ernesto Che Guevara · 25000 Constantine

Contents

PART I: THE BIG PICTURE

1 How to read this guide

2 The platform on one page

3 The dev stack, and why each piece

4 Map of the repository

PART II: THE BACKEND, IN CODE

5 Life of a request

6 Modules and routing order

7 The data model

8 One store per deployment, and caching

9 Security

10 Orders: the state machine

11 The stock engine

12 Invoices

13 Money and reports

14 The Excel import wizard

15 Cover pictures

16 Packs (plans)

17 Arabic search

PART III: THE INTERFACES

18 The storefront

19 The admin dashboard

20 The mobile apps

21 Editions: one codebase, two products

PART IV: RUNNING IT

22 Deployment and operations

23 Testing and quality

PART V: WHY IT WORKS, AND WHAT'S NEXT

24 What makes this platform good

25 Roadmap

26 Glossary

1. How to read this guide

Each chapter starts in plain words: what the part does and why it exists. Then it shows the real code behind it. The code boxes use three visual styles:

path/to/file.ts — where the code lives in the repository

```
// Grey italic: a comment that is already in the source code.
const answer = 42;   // normal colour = the real code
```

A gold note is added for this guide only. It explains the line(s) above it in plain words. It is not in the source file.

Excerpts are trimmed to the part that matters. A line with `// …` means code was skipped. Paths are relative to the repository root, so you can open the file and read the rest.

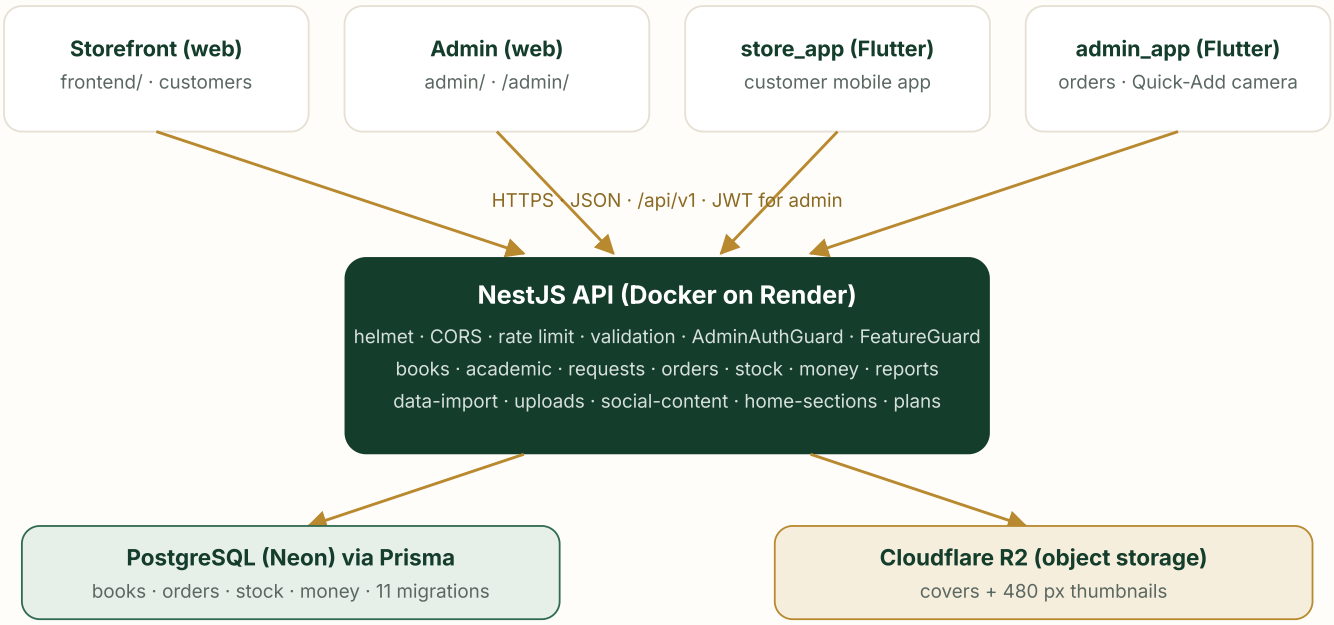
> **Who is this for?** You, the owner and developer. The Arabic دليل المنصة (`docs/platform-guide.pdf`) explains the features for shop staff and buyers. This guide explains how those features are built.

Generated from the repository at commit `749627f` (November 2026). The source is `docs/guide/technical-guide.html`: edit it and print it again to update this PDF.

PART I · CHAPTER 2

The platform on one page

One API and one database serve four apps. All four are in Arabic (right to left) and share the same green-and-gold identity.



Storefront: المتجر

Home page laid out by the admin, instant search, a page for each book, categories, collections (shelves), and orders and book requests that end in WhatsApp. Customers don't need an account.

Admin: لوحة التحكم

Orders, counter sales (POS), books, stock, purchases and suppliers, expenses, reports, Excel import and export, social-media export, store settings and packs.

Mobile store app

The storefront as a native Android/iOS app. It reads the same public endpoints.

Mobile admin app

The back office in your pocket. Its standout feature is **Quick-Add**: photograph a cover, type the title, save, and scan the next book.

The features at a glance

Area	What it does	Where in code
Catalogue	Books with authors, publishers, category, country, year, price, cost price, gallery, notes. Academic tree (field → year → subject).	`books/`, `academic/`
Storefront layout	Admin reorders, renames or hides home blocks, and builds hand-picked shelves.	`home-sections/`

Area	What it does	Where in code
Leads	"Request a book" form → admin inbox → can become an order.	`requests/`
Sales register	Website orders, phone orders, counter sales and imported sales in one table.	`admin/orders/`
Stock	Moves automatically with sales, purchases, returns and imports. Every move is logged.	`common/stock/`
Invoices	Numbered `2026-0001`, with the legal IDs (RC/NIF/NIS/ART), printable.	`orders/:id/invoice`
Money	Suppliers, purchases (stock in), payments and balance, expenses.	`admin/money/`
Reports	Revenue, cost of goods sold, margin, expenses, net profit, top and slow books, stock value.	`admin/reports/`
Excel	Import a shop's own spreadsheets (catalogue or sales/purchase/expense history) with preview and undo. Exports.	`admin/data-import/`
Covers	Upload → normalised JPEG + thumbnail in R2. Matching phone photos to books by OCR.	`uploads/`, `image-sync/`
Social content	ZIP of covers + captions + metadata, ready to post.	`social-content/`
Packs	Starter / Pro / Business: features locked or unlocked, with limits.	`common/plan/`

PART I · CHAPTER 3

The dev stack, and why each piece

Every layer is a mainstream, well-documented choice. That keeps the platform easy to hire for, easy to host cheaply, and easy for an AI assistant to work on safely.

Layer	Technology	Why it was chosen
Language	TypeScript 6, everywhere on the web	One language for server and browsers. Types catch mistakes (a missing field, a wrong status) before they reach a customer.
API framework	NestJS 11 on Express 5	Gives the server a fixed shape: *modules* group a feature, *controllers* receive HTTP calls, *services* hold the logic, *guards* decide who may enter. Every feature looks the same, so the code stays readable as it grows.
Database	PostgreSQL (Neon in production)	Reliable transactions (a sale and its stock move succeed or fail together), exact `Decimal` money, and powerful SQL for reports.
ORM	Prisma 5 (`relationJoins`)	The schema file is the single source of truth. Prisma generates typed queries and versioned migrations. `relationJoins` turns an `include` into one SQL query instead of one per relation.
Validation	class-validator DTOs + global `ValidationPipe`	Every incoming body is checked and cleaned before any code runs. Unknown fields are dropped (`whitelist`).
Auth	JWT (`@nestjs/jwt`)	Stateless 30-day admin token. Nothing to store on the server.
Hardening	helmet, @nestjs/throttler, CORS allowlist	Security headers, per-IP rate limits, and only known origins may call the API from a browser.
Files	Cloudflare R2 (S3 API) + sharp	R2 has no download (egress) fees, so covers are cheap to serve. sharp shrinks phone photos about 20× and strips GPS data.
Excel	SheetJS (`xlsx` 0.20.3 from the official CDN)	Reads .xlsx, .xls and .csv. The CDN build is the patched one; the npm copy is stuck on an old, vulnerable version.
OCR	tesseract.js	Reads cover text to match phone photos to books, with no paid API.
Web UI	React 19 + Vite 8	Instant dev reloads and small, hashed production bundles.
Styling	Tailwind CSS 4 + Base UI primitives (shadcn style)	Accessible dialogs, menus and focus handling, with styles written next to the markup. RTL works with logical properties.
Data fetching	TanStack Query 5 + axios	Caching, background refresh and loading/error states for free. Pages stay simple.

Layer	Technology	Why it was chosen
Routing	React Router 7	Pages, redirects, and lazy-loaded admin screens.
Mobile	Flutter (separate repo)	One codebase for Android and iOS. Talks to the same API over HTTP.
Hosting	Render Blueprint (`render.yaml`) + Docker	The API, storefront and admin are deployed from one file. Pushing to `master` redeploys.
Tests	Jest + ts-jest	Fast unit tests for the pure business rules: money, stock statuses, parsing.

> **Design rule behind the stack:** no exotic tools. Each piece is the most common way to do that job in 2026. Any developer you hire, and Claude, already knows them.

PART I · CHAPTER 4

Map of the repository

Where everything lives. Each top-level folder has one job.

```
bookstore/
├── src/                          the NestJS API
│   ├── main.ts                   boot: security headers, CORS, validation, start listening
│   ├── app.module.ts             wires every module together (order matters!)
│   ├── common/                   shared building blocks used by many modules
│   │   ├── guards/               AdminAuthGuard: "is this a valid admin token?"
│   │   ├── tenant/               StoreResolver: "which store is this deployment?"
│   │   ├── stock/                moveStock(): the only way stock changes from money flows
│   │   ├── plan/                 packs: features, limits, @RequiresFeature
│   │   ├── home/                 the storefront home-layout block kinds
│   │   ├── validation/           query-string checks → 400, never 500
│   │   └── utils/                covers (sharp), safe image fetch, ZIP writer, Arabic normaliser…
│   ├── modules/                  one folder per feature
│   │   ├── books/ academic/ requests/ images/          public storefront API
│   │   ├── import/ image-sync/ clean/                  operational tools (admin-only)
│   │   ├── storage/                                    Cloudflare R2 wrapper
│   │   └── admin/                                      the back office, /api/v1/admin/*
│   │       ├── auth/ stats/ orders/ books/ catalog/ academic/ inventory/
│   │       ├── uploads/ data-import/ money/ reports/
│   │       └── home-sections/ social-content/
│   └── prisma/                   PrismaService (the database connection)
├── prisma/
│   ├── schema.prisma             THE data model: 27 tables + 7 enums
│   └── migrations/               11 versioned SQL migrations, applied on boot
├── frontend/                     public storefront (React + Vite) · :5173
│   └── src/ pages/ components/ lib/
├── admin/                        admin dashboard (React + Vite) · :5175/admin/
│   └── src/ pages/admin/ components/admin/ lib/
├── scripts/                      one-off tools: set-plan, setup-store, backfill-thumbnails, perf/
├── docs/                         guides (this one), product plan, sales kit
├── Dockerfile                how the API image is built for production
├── render.yaml               deploys API + storefront + admin on Render
└── CLAUDE.md                 project briefing for Claude Code
```

Conventions that repeat everywhere

A feature module looks like this

- `x.module.ts` : declares the pieces
- `x.controller.ts` : URLs → method calls
- `x.service.ts` : the logic and the database calls
- `dto/` : shapes of incoming bodies, validated
- `*-rules.ts` / `*.spec.ts` : pure logic + its tests

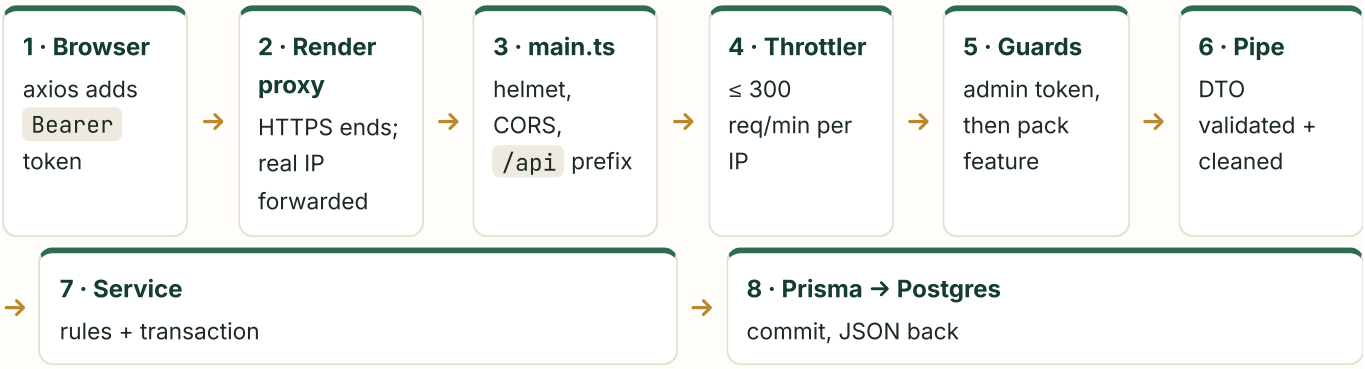
Rules of thumb

- Every admin query filters by `storeId` .
- Money is `Decimal(10,2)` , rounded to the cent once.
- Stock changes only through `moveStock()` (or a logged hand adjustment).
- Public routes never return `costPrice` .
- Errors shown to staff are in Arabic.

PART II · CHAPTER 5

Life of a request

Take a staff member clicking "Confirm" on an order. Here is every stop that click passes through.



Steps 3 and 6 are configured once, at boot, in `main.ts`:

src/main.ts — server start-up

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
```

▸ Build the whole app from the root module (AppModule, next chapter): every controller, service and guard.

```
  app.set('trust proxy', Number(process.env.TRUST_PROXY_HOPS ?? 1));
```

▸ Render sits in front of the API. Without this line, every visitor would look like the same IP (Render's), so one busy customer could use up everyone's rate limit. "Trust one hop" means: read the real visitor's IP from the proxy header.

```
  app.use(
    helmet({
      crossOriginResourcePolicy: { policy: 'cross-origin' },
    }),
  );
```

▸ helmet adds about a dozen security headers to every response (blocks click-jacking, MIME sniffing…). Cover images are loosened to "cross-origin" because the storefront, on another domain, displays them.

```
  const corsOrigins = process.env.CORS_ORIGINS?.split(',').map((s) => s.trim()).filter(Boolean);
  app.enableCors({
    origin: corsOrigins?.length ? corsOrigins : false,
    credentials: true,
    exposedHeaders: ['Content-Disposition', 'X-Export-Books', 'X-Export-Truncated'],
  });
```

▸ CORS: only the origins listed in CORS_ORIGINS (your storefront and admin URLs) may call the API from a browser. With no list, cross-origin calls are refused, which is the safe default. "exposedHeaders" lets the admin read the file name of an Excel/ZIP export.

```
  app.setGlobalPrefix('api');
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
```

▸ Every route starts with /api. The ValidationPipe checks each body against its DTO class: wrong types → 400 error; unknown fields are dropped (whitelist); "transform" turns the JSON into the DTO class so clean-up rules (e.g. phone-number formatting) really run.

```
  app.useGlobalFilters(new AllExceptionsFilter());
```

▸ One place that turns every error into the same JSON shape { statusCode, error, message, path, timestamp } and logs real server errors.

```

  app.enableShutdownHooks();
```

▸ During a deploy Render sends SIGTERM: finish requests in flight and close the database pool cleanly instead of cutting them.

```

  const port = process.env.PORT ?? 3000;
  await app.listen(port);
  keepAwake();
```

▸ Start listening, then start the "keep awake" ping (chapter 22) so the free Render plan never puts the API to sleep.

```
}
```

PART II · CHAPTER 6

Modules and routing order

`app.module.ts` is the table of contents of the server. It plugs every feature in, and the order it lists them in matters.

src/app.module.ts — root module

```
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
```

▸ Reads the .env file once and makes the values (DATABASE_URL, R2_*, ADMIN_PASSWORD...) available everywhere.

```
    ThrottlerModule.forRoot([{ ttl: 60_000, limit: 300 }]),
```

▸ Rate limit: 300 requests per 60 seconds per IP. Individual routes tighten it (login 10/min, request form 5/min).

```
    AuthModule,
    StorageModule,
    ServeStaticModule.forRoot({ rootPath: join(process.cwd(), 'public'), serveRoot: '/',
serveStaticOptions: { index: false } }),
```

▸ Serves files from /public (local covers in development, when R2 isn't configured).

```
    PrismaModule,
    TenantModule,
    // AdminModule first: its literal routes (v1/admin/books) must register
    // before the public wildcard routes (v1/:storeSlug/books), otherwise
    // ":storeSlug" swallows "admin" and admin GETs 404 with "Store not found".
    AdminModule,
```

▸ The most important line in the file. Public routes look like /v1/:storeSlug/books, where :storeSlug can be ANY word, including "admin". Express matches routes in the order they were added, so the admin routes must be added first or /v1/admin/books would be read as "store named admin".

```
    BooksModule,
    RequestsModule,
    AcademicModule,
    ImportModule,
    ImagesModule,
    CleanModule,
    ImageSyncModule,
  ],
  controllers: [HealthController],
```

▸ GET /api/v1/health returns {status, db, uptime}. Render calls it to know the API is alive.

```
  providers: [{ provide: APP_GUARD, useClass: ThrottlerGuard }],
```

▸ Turns the rate limiter on for every route in the app.

```
})
export class AppModule {}
```

The public API vs. the admin API

Public: `/api/v1/:storeSlug/…`

- `GET books`, `books/:id`, search, recommendations
- `GET home-sections` (the home layout)
- `GET academic/…` (fields, years, subjects)
- `POST requests` (5/min per IP)

No login. List endpoints return the slim "card" shape. Only `books/:id` returns everything.

Admin: `/api/v1/admin/…`

- `POST auth/login` (10/min) → JWT
- orders, books, catalog, inventory, uploads, data-import, money, reports, home-sections, social-content, stats, plan

Every controller carries `@UseGuards(AdminAuthGuard)`. The paid features also carry `@RequiresFeature(…)`.

PART II · CHAPTER 7

The data model

`prisma/schema.prisma` describes every table. Prisma turns it into SQL migrations and into the typed client that the services use.

Group	Tables
Store	`Store` (name, slug, plan, legal IDs), `StoreCounter` (invoice sequence), `HomeSection` + `HomeSectionBook`
Catalogue	`Book`, `BookImage` (gallery), `Author` / `BookAuthor`, `Publisher` / `BookPublisher`, `Category`, `Country`
Academic	`Field` → `AcademicYear` → `Subject`, with link tables `BookOnField/Year/Subject`
Stock	`Inventory` (count + status per book), `StockMovement` (log of every change and why)
Sales	`Request` (lead), `Order`, `OrderItem`
Money	`Supplier`, `Purchase`, `PurchaseItem`, `Expense`
Imports	`ImportJob`: everything an import created is tagged with its id, so the import can be undone

Authors, publishers, categories, countries and the academic tree are **shared** reference tables. Everything else belongs to one store through `storeId`.

prisma/schema.prisma — model Order (excerpt)

```
model Order {
  id      String @id @default(uuid())
  storeId String
```

▸ Every order belongs to one store. All admin queries filter on it.

```
  // Customer snapshot (denormalized — orders survive even if customer info changes elsewhere)
  firstName String
  lastName  String
  phone     String
  wilaya    String
  address   String
```

▸ The customer's details are copied INTO the order. If the same customer orders again with a new address, old orders still show what was true at the time.

```
  status OrderStatus @default(pending)
  channel       SaleChannel    @default(online)
  paymentMethod PaymentMethod?
```

▸ channel = online | shop | import. This one field is what lets website orders, counter sales and imported history live in a single sales register.

```
  // Money — denormalized totals so historical orders aren't affected by price changes
  subtotal     Decimal @default(0) @db.Decimal(10, 2)
  discount     Decimal @default(0) @db.Decimal(10, 2)
```

```
  shippingCost Decimal @default(0) @db.Decimal(10, 2)
  total        Decimal @default(0) @db.Decimal(10, 2)
```

▸ Decimal(10,2) stores exact cents. Floats would turn 0.1 + 0.2 into 0.30000000000000004, which is unacceptable for money. Totals are frozen too: raising a book's price tomorrow does not change yesterday's invoice.

```
  stockApplied Boolean @default(false)
```

▸ A safety flag: "have this order's books already left the shelf?" It makes sure stock is taken out exactly once and put back exactly once, however many times the status changes.

```
  invoiceNumber String?
  invoicedAt    DateTime?
  soldAt DateTime?
```

▸ soldAt = the real date of an imported old sale (createdAt would be the import day). Reports use soldAt when it exists.

```
  importJobId String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  // …
}
```

prisma/schema.prisma — model Book (excerpt)

```
model Book {
  id              String  @id @default(uuid())
  title           String
  titleNormalized String?
```

▸ A copy of the title with Arabic spelling variants flattened (chapter 17). Search compares against this.

```
  price     Decimal? @db.Decimal(10, 2)
  /// What the store paid for it. Admin-only — never sent to the storefront.
  costPrice Decimal? @db.Decimal(10, 2)
```

▸ The purchase price feeds the margin and profit reports. Public routes pass every book through serializePublicBook(), which deletes this field before the JSON leaves the server.

```
  importJobId String?
  categoryId String
  storeId    String
  imageUrl String?
```

▸ The main cover. It is also picture 0 of the gallery (BookImage). setBookCover() keeps the two in step.

```
  // …
}
```

PART II · CHAPTER 8

One store per deployment, and caching

The database can hold several stores. Each deployment, though, serves one store, and `StoreResolver` answers "which one?" quickly.

> **Why this matters for selling the platform:** the data is already multi-store, with a `storeId` on every row. To sell to a second shop today, you deploy a second copy pointed at its own store. Running many stores on one deployment needs per-user accounts first (chapter 25).

src/common/tenant/store-resolver.service.ts — tenancy + caches

```
@Injectable()
export class StoreResolver {
  private cached: { id: string; slug: string } | null = null;

  async getStore() {
    if (this.cached) return this.cached;
```

▸ Asked on almost every admin request. After the first lookup the answer is kept in memory forever (the store of a deployment never changes).

```
    const slug = process.env.STORE_SLUG;
    const store = slug
      ? await this.prisma.store.findUnique({ where: { slug } })
      : await this.prisma.store.findFirst({ orderBy: { createdAt: 'asc' } });
```

▸ Which store? The one named by STORE_SLUG in the environment, else the oldest store in the database.

```
    if (!store) throw new NotFoundException('No store configured');
    this.cached = { id: store.id, slug: store.slug };
    return this.cached;
  }

  async getPlan(): Promise<string> {
    if (this.planCache && Date.now() - this.planCache.at < 60_000) return this.planCache.plan;
```

▸ The store's pack is re-read at most once a minute. A pack change (scripts/set-plan.ts) shows up within 60 s without a database hit on every request.

```
    // …
  }

  async bySlug(slug: string): Promise<{ id: string; slug: string; name: string }> {
    const hit = this.bySlugCache.get(slug);
    if (hit && Date.now() - hit.at < SLUG_TTL_MS) {
      if (!hit.store) throw new NotFoundException('Store not found');
      return hit.store;
    }
```

▸ Every storefront request starts by turning the slug in the URL ("elbayan") into a store. Caching it for one minute saves one database round trip (about 50 ms in production) on every page view. Unknown slugs are cached too, so a bot can't force repeated lookups.

```
    const store = await this.prisma.store.findUnique({ where: { slug }, select: { id: true, slug:
true, name: true } });
    if (this.bySlugCache.size ⩾ 100) this.bySlugCache.clear();
```

▸ A hard cap: crawlers trying random slugs cannot grow this map forever and eat memory.

```
    this.bySlugCache.set(slug, { store, at: Date.now() });
    if (!store) throw new NotFoundException('Store not found');
    return store;
```

```
  }
}
```

PART II · CHAPTER 9

Security

Security is layered: each layer assumes the one before it might fail.

Layer	What it stops
HTTPS + helmet	Eavesdropping, click-jacking, content-type tricks
CORS allowlist	Other websites calling your API from a visitor's browser
Rate limits	Password guessing (login 10/min), form spam (5/min), scraping and floods (300/min)
AdminAuthGuard	Anyone without a valid admin JWT, and a valid token aimed at another store
ValidationPipe + query checks	Malformed or oversized input, unknown fields, bad enum values (→ 400, never a crash)
storeId on every query	Reading or changing another store's rows
Public serializer	Leaking `costPrice` and other internal fields to the storefront
Upload checks	Fake images: the file type is decided by its first bytes, not its name. Decompression bombs are refused.
Safe image fetch	Tricking the server into fetching internal addresses (SSRF)
Excel export safety	Cells starting with `=`, `+`, `-`, `@` are escaped, so an export can't run formulas in Excel
Production boot check	The API refuses to start in production without `JWT_SECRET`

src/common/guards/admin-auth.guard.ts — who may enter the back office

```
async canActivate(context: ExecutionContext): Promise<boolean> {
  const req = context.switchToHttp().getRequest<Request>();
  const header = req.headers.authorization ?? '';
  const [scheme, token] = header.split(' ');
  if (scheme !== 'Bearer' || !token) {
    throw new UnauthorizedException('Missing admin token');
  }
```

▸ Step 1: there must be an "Authorization: Bearer <token>" header. No header → 401 (the admin app then sends you to the login page).

```
  try {
    const payload = await this.jwt.verifyAsync<{ role?: string }>(token);
    if (payload.role !== 'admin') throw new Error('not admin');
  } catch {
    throw new UnauthorizedException('Invalid or expired admin token');
  }
```

▸ Step 2: the token's signature is checked with JWT_SECRET, plus its 30-day expiry and its role. A forged or expired token fails here. "Fails closed": any doubt means no entry.

```
  const slug = req.params?.storeSlug;
  if (slug !== undefined) {
```

```
    const own = await this.stores.getStore();
    if (slug !== own.slug) {
      throw new ForbiddenException('This admin account cannot access that store');
    }
  }
```

▸ Step 3 (tenancy): some admin tools have the store in the URL (/v1/:storeSlug/import). Editing the slug to another store's name is refused with 403, even with a valid token.

```
  return true;
}
```

src/common/utils/image-source.ts — fetching pictures from the internet safely

```
async function isPublicHttps(url: URL): Promise<boolean> {
  if (url.protocol !== 'https:' || url.username || url.password) return false;
  const host = url.hostname.replace(/^\[|\]$/g, '');
  if (!host || host === 'localhost' || host.endsWith('.localhost') || host.endsWith('.internal'))
return false;
```

▸ The social export downloads covers from URLs stored in the database, and those URLs could have been typed by anyone. Only plain https links are allowed.

```
  let addresses: string[];
  // …
  addresses = (await dns.lookup(host, { all: true, verbatim: true })).map((a) => a.address);
  return addresses.length > 0 && addresses.every((a) => !isPrivateAddress(a));
```

▸ The name is resolved to its real IP addresses, and EVERY one must be public. Otherwise an attacker could register "evil.com → 127.0.0.1" and make your server read its own internal services (SSRF). Redirects are re-checked the same way (at most 3), and a download is cut at 15 MB.

```
}

export function isPrivateAddress(ip: string): boolean {
  if (net.isIPv4(ip)) {
    const [a, b] = ip.split('.').map(Number);
    return (
      a === 0 || a === 10 || a === 127 ||
      (a === 100 && b >= 64 && b <= 127) || // CGNAT
      (a === 169 && b === 254) || // link-local, cloud metadata
      (a === 172 && b >= 16 && b <= 31) ||
      (a === 192 && b === 168) ||
      // …
      a >= 224 // multicast + reserved
    );
  }
```

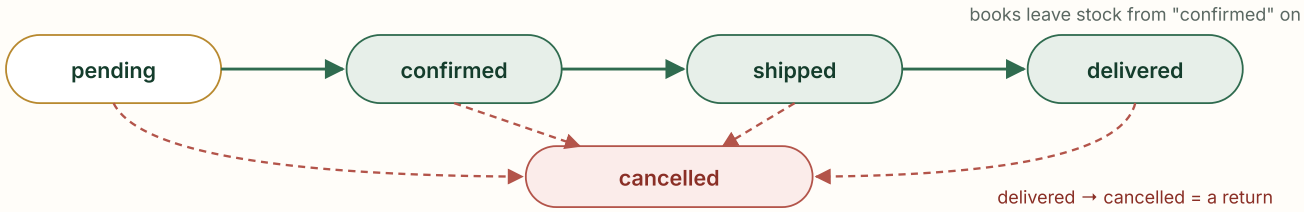
▸ The list of address ranges that never belong to a public website: your own machine (127.x), home and office networks (10.x, 192.168.x), and the cloud "metadata" address (169.254.x) that holds server secrets.

```
  // … IPv6 equivalents
}
```

PART II · CHAPTER 10

Orders: the state machine

An order moves through fixed steps. The rules live in a small file with no database code, so they can be tested on their own.



src/modules/admin/orders/order-rules.ts — pure business rules

```ts
export const ORDER_TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
  pending: ['confirmed', 'cancelled'],
  confirmed: ['shipped', 'cancelled'],
  shipped: ['delivered', 'cancelled'],
  // A delivered sale can still be returned: cancelling it puts the books back
  // in stock and takes it out of the income.
  delivered: ['cancelled'],
  cancelled: [],
};
```

The whole workflow as a lookup table: for each status, where it may go next. "cancelled" is final. Changing the workflow later means editing this table and nothing else.

```ts
export function canTransition(from: OrderStatus, to: OrderStatus): boolean {
  if (from === to) return true;
  return ORDER_TRANSITIONS[from]?.includes(to) ?? false;
}
```

Asked before every status change. Jumping "pending → delivered" is refused with a clear error.

```ts
const cents = (n: number) => Math.round(n * 100) / 100;

export function computeOrderTotals(items: { unitPrice: number; quantity: number }[], shipping:
number, discount = 0): OrderTotals {
  const subtotal = cents(items.reduce((s, it) => s + it.unitPrice * it.quantity, 0));
  const d = cents(Math.min(Math.max(0, discount || 0), subtotal));
  const shippingCost = cents(Math.max(0, shipping || 0));
  return { subtotal, discount: d, shippingCost, total: cents(subtotal - d + shippingCost) };
}
```

The ONLY place order money is computed. Subtotal = price × quantity per line. The discount is clamped: never negative, never more than the books are worth. Shipping is never negative. Everything is rounded to the cent here, once.

src/modules/admin/orders/orders.service.ts — updateStatus(): where rules meet the database

```ts
async updateStatus(id: string, target: OrderStatus) {
  const storeId = await this.storeResolver.getStoreId();
  const order = await this.prisma.order.findFirst({ where: { id, storeId }, include: { items: true }
});
  if (!order) throw new NotFoundException('Order not found');
```

"where: { id, storeId }": an order id from another store is simply "not found".

```ts
  if (!canTransition(order.status as OrderStatus, target)) {
```

```
    throw new BadRequestException(`Cannot transition order from ${order.status} to ${target}`);
  }

  return this.prisma.$transaction(async (tx) => {
```

Everything inside $transaction succeeds together or is rolled back together. A status can never change while its stock move fails, or the other way round.

```
    // Conditional update: two clicks at once cannot move stock twice.
    const moved = await tx.order.updateMany({ where: { id, status: order.status }, data: { status:
target } });
    if (moved.count === 0) throw new BadRequestException('تغيّرت حالة الطلب للتو. أعد تحميل الصفحة.');
```

Protection against double clicks and two staff members at once: the update only matches if the status is STILL what we read a moment ago. If someone else changed it first, 0 rows match and we stop with "the order just changed, reload".

```
    const lines = order.items.map((i) => ({ bookId: i.bookId, quantity: i.quantity }));
    if (takesStock(target) && !order.stockApplied) {
      await this.applyStock(tx, storeId, id, lines, -1);
```

Moving into confirmed/shipped/delivered for the first time → books leave stock, and stockApplied becomes true.

```
    } else if (target === 'cancelled' && order.stockApplied) {
      await moveStock(tx, storeId, lines.map((l) => ({ bookId: l.bookId, delta: l.quantity })),
'sale_cancel', id);
      await tx.order.update({ where: { id }, data: { stockApplied: false } });
```

Cancelling (or returning) an order whose books had left → they come back, logged as "sale_cancel".

```
    }
    return tx.order.findUnique({ where: { id }, include: orderInclude });
  });
}
```

PART II · CHAPTER 11

The stock engine

Sales, returns, purchases, purchase cancellations and history imports all change stock through one function. That's why the numbers stay right.

src/common/stock/stock.ts — moveStock()

```
export async function moveStock(
  tx: Tx,
  storeId: string,
  lines: StockLine[],
  reason: StockReason,
  refId?: string | null,
  note?: string | null,
): Promise<void> {
```

tx = the caller's open transaction. A stock move is never on its own; it is part of the sale or purchase that caused it. reason = sale | sale_cancel | purchase | purchase_cancel | import | adjustment... refId = the order or purchase id, so the log links back.

```
  const net = new Map<string, number>();
  for (const l of lines) {
    if (!l.bookId || !Number.isInteger(l.delta) || l.delta === 0) continue;
    net.set(l.bookId, (net.get(l.bookId) ?? 0) + l.delta);
  }
```

If the same book appears on two lines (e.g. 2 + 1 copies), merge them into one net change (−3). Junk lines are ignored.

```
  for (const [bookId, delta] of entries) {
    const updated = await tx.$queryRaw<{ stock: number | null; status: InventoryStatus }[]>`
      UPDATE "Inventory"
         SET "stock" = CASE
               WHEN "stock" IS NULL AND ${delta}::int > 0 THEN ${delta}::int
               WHEN "stock" IS NULL THEN NULL
               ELSE GREATEST(0, "stock" + ${delta}::int)
             END,
             "updatedAt" = now()
       WHERE "bookId" = ${bookId} AND "storeId" = ${storeId}
   RETURNING "stock", "status"`;
```

One atomic SQL statement does read + add + write. Two cashiers selling the LAST copy at the same instant can't both read "1"; the database runs them one after the other. The rules: an unknown count (NULL) stays unknown on a sale (we can't invent a number) but becomes known on a delivery. GREATEST(0, ...) means stock never goes below zero. ${...} values are sent as parameters, so there's no SQL injection.

```
    if (!updated.length) {
      if (delta > 0) {
        await tx.inventory.create({ data: { bookId, storeId, stock: delta, status: 'available' } });
      }
      continue;
    }
```

The book had no stock row yet: a delivery creates one; a sale leaves it untracked.

```
    const { stock, status } = updated[0];
    let next: InventoryStatus | null = null;
    if (stock === 0 && status === 'available') next = 'on_request';
    if (stock !== null && stock > 0 && status === 'on_request') next = 'available';
    if (next) await tx.inventory.update({ where: { bookId }, data: { status: next } });
```

▸ The storefront label follows the count: the last copy sold → "on request"; copies arrive → "available" again. "Rare" is a choice the owner made, so it is never changed automatically.

```
  }

  await tx.stockMovement.createMany({
    data: entries.map(([bookId, delta]) => ({ storeId, bookId, delta, reason, refId: refId ?? null,
note: note ?? null })),
  });
```

▸ Every change is written to the StockMovement log: book, +/− amount, reason, linked document, time. The admin shows it as the stock history of each book, so you can always answer "where did this copy go?".

```
}
```

Rule: any new feature that sells, buys or returns books must call `moveStock()` inside its transaction. Hand edits on the Inventory screen write an `adjustment` movement, so they're logged too.

PART II · CHAPTER 12

Invoices

An invoice number must be unique, sequential within the year, and must never change once printed. That's what Algerian accounting expects.

src/modules/admin/orders/orders.service.ts — invoice()

```
async invoice(id: string) {
  const store = await this.storeResolver.getStore();
  let order = await this.prisma.order.findFirst({ where: { id, storeId: store.id }, include: { items:
true } });
  if (!order) throw new NotFoundException('Order not found');
  if (order.status === 'cancelled') throw new BadRequestException('لا تُصدر فاتورة لطلب ملغى.');
```

▸ No invoice for a cancelled order.

```
  if (!order.invoiceNumber) {
```

▸ Only numbered the FIRST time it's printed. Reprinting shows the same number.

```
    const year = new Date().getFullYear();
    order = await this.prisma.$transaction(async (tx) => {
      const counter = await tx.storeCounter.upsert({
        where: { storeId_key: { storeId: store.id, key: `invoice-${year}` } },
        create: { storeId: store.id, key: `invoice-${year}`, value: 1 },
        update: { value: { increment: 1 } },
      });
```

▸ A counter row per store per year ("invoice-2026"). upsert = create it at 1 on the first invoice of the year, otherwise add 1. The database does the increment atomically, so two invoices can never get the same number. A new year starts again at 0001 automatically.

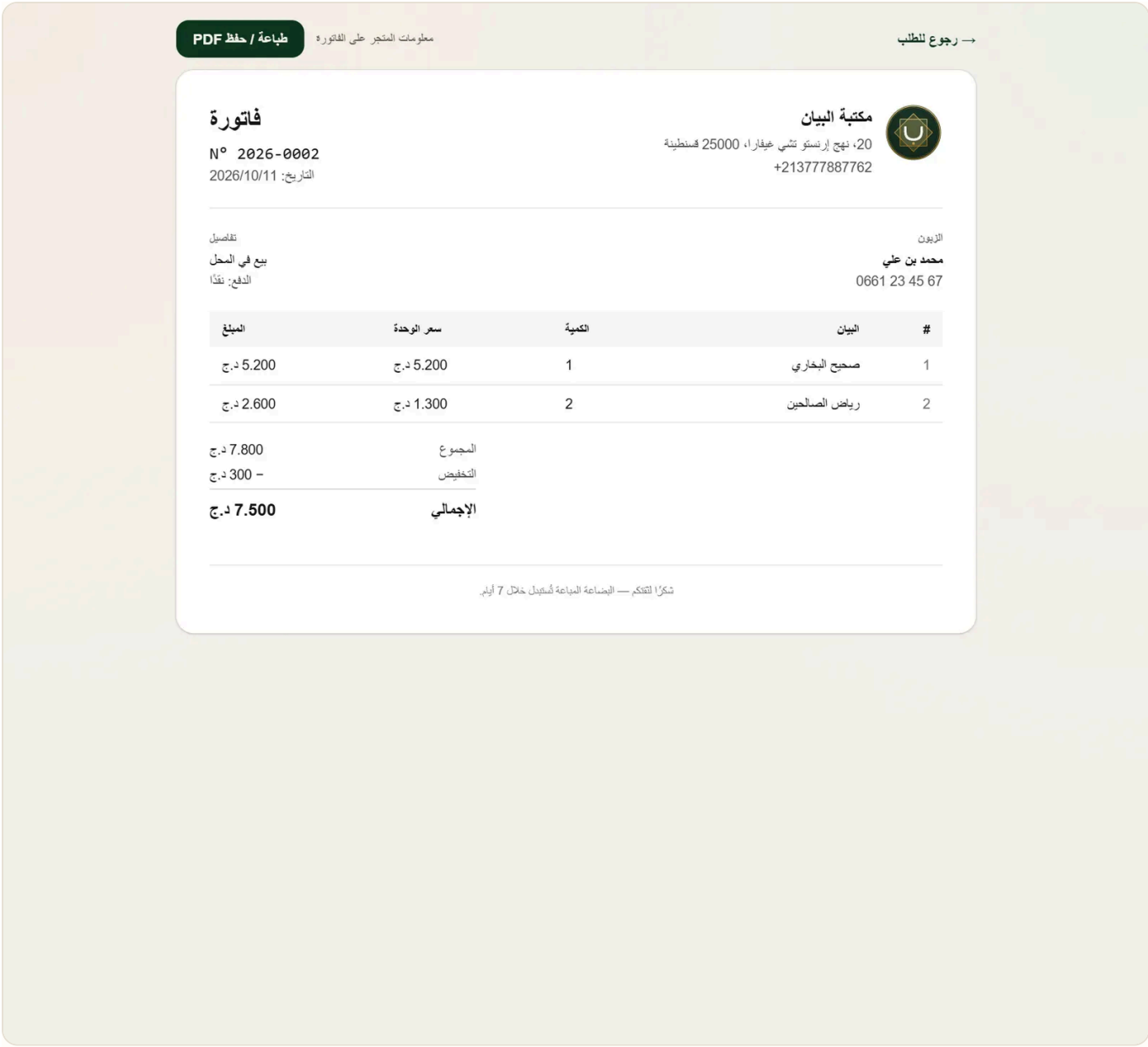
```
      // Only the first caller numbers the invoice; a parallel one keeps it.
      await tx.order.updateMany({
        where: { id, invoiceNumber: null },
        data: { invoiceNumber: `${year}-${String(counter.value).padStart(4, '0')}`, invoicedAt: new
Date() },
      });
```

▸ Writes "2026-0007" only if the order still has no number. If two tabs print at the same moment, the second one keeps the first one's number.

```
      return tx.order.findUniqueOrThrow({ where: { id }, include: { items: true } });
    });
  }
  const profile = await this.prisma.store.findUniqueOrThrow({
    where: { id: store.id },
    select: { name: true, phone: true, address: true, email: true, rc: true, nif: true, nis: true,
art: true, invoiceFooter: true },
  });
```

▸ The shop's legal identity (RC, NIF, NIS, ART) from Settings is printed on every invoice.

```
  // …
}
```



The printable invoice page (`/admin/invoice/:id`). It sits outside the admin shell so it prints cleanly on A4 or saves as PDF.

PART II · CHAPTER 13

Money and reports

The money module records what comes in and goes out. The reports module turns it into the numbers a shop owner actually asks for.

Purchases

Buying from a supplier adds the books to stock (`moveStock 'purchase'`) and updates each book's `costPrice`. Payments are recorded against it, so you know what you still owe. Cancelling it removes the stock again.

Expenses

Rent, salaries, transport, electricity… each with a category, so the report can show where the money goes.

Frozen cost

When a book is sold, its current `costPrice` is copied onto the order line as `unitCost`. Later purchase prices never rewrite the margin of past sales.

src/modules/admin/reports/reports.service.ts — overview(): revenue and cost of goods per month

```ts
/** Orders that count as income: confirmed, shipped or delivered. */
const SOLD = Prisma.sql`('confirmed','shipped','delivered')`;
/** A sale's date: when it happened (imports), else when it was recorded. */
const SALE_DATE = Prisma.sql`COALESCE(o."soldAt", o."createdAt")`;
```

▸ Two definitions used by every query, so all reports agree on what "a sale" is and when it happened. Pending and cancelled orders are never income.

```ts
const [sales, costs, /* … 8 more */] = await Promise.all([
```

▸ Ten independent queries are sent to the database at the SAME time, not one after another. The report page loads in about the time of the slowest query.

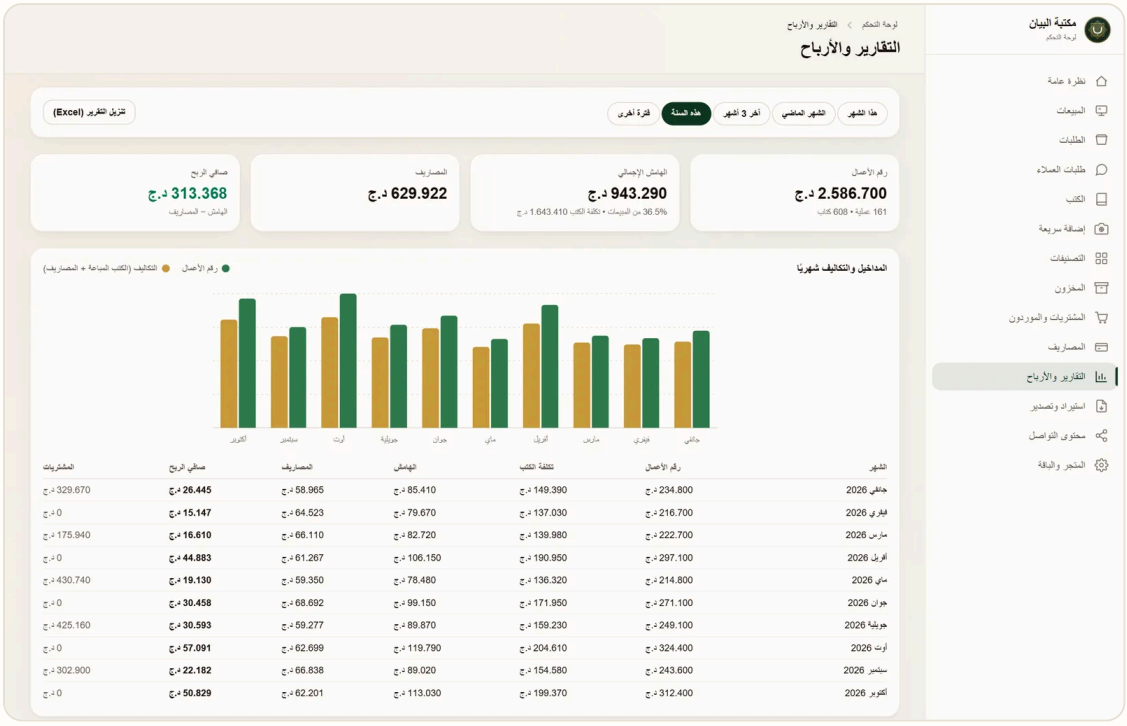
```ts
  this.prisma.$queryRaw`
    SELECT to_char(date_trunc('month', ${SALE_DATE}), 'YYYY-MM') AS month,
           count(*) AS orders,
           sum(o.total) AS revenue,
           sum(o.subtotal - o.discount) AS book_revenue,
           sum(o."shippingCost") AS shipping
      FROM "Order" o
     WHERE o."storeId" = ${storeId} AND o.status::text IN ${SOLD}
       AND ${SALE_DATE} BETWEEN ${from} AND ${to}
     GROUP BY 1 ORDER BY 1`,
```

▸ Revenue per month, split into books vs. shipping. Raw SQL is used because the database can sum thousands of orders far faster than looping over them in JavaScript.

```ts
  this.prisma.$queryRaw`
    SELECT to_char(date_trunc('month', ${SALE_DATE}), 'YYYY-MM') AS month,
           sum(i.quantity) AS books,
           sum(CASE WHEN i."unitCost" IS NOT NULL THEN i.quantity * i."unitCost" ELSE 0 END) AS cogs,
           sum(CASE WHEN i."unitCost" IS NOT NULL THEN i.quantity * i."unitPrice" ELSE 0 END) AS
costed_revenue,
           sum(i.quantity * i."unitPrice") AS line_revenue
      FROM "OrderItem" i JOIN "Order" o ON o.id = i."orderId"
     -- …same filters…
```

COGS = cost of goods sold, from the frozen unitCost. "costed_revenue" counts only lines whose cost is known, so the margin % is honest: a book with no purchase price doesn't look like a 100 % profit. The report also says how much revenue has no known cost.

```
  // … expenses by month and category, purchases, channels, top books, slow stock (no sale in 90 days), stock value
]);
```



Reports: revenue, cost of goods, margin, expenses and net profit per month, plus channels, top sellers and slow stock. Exportable to Excel.

PART II · CHAPTER 14

The Excel import wizard

Shops arrive with years of hand-typed spreadsheets. The wizard reads them as they are: Arabic or French headers, prices like «1 500 دج», Arabic-Indic digits, Hijri years.

1 · Analyze
Upload. The sheet is read, the layout detected, columns recognised by synonyms (`fields.ts`). Kept 30 min in memory.

→

2 · Map
Admin checks which column is which. The suggested kind: catalogue, sales, purchases or expenses.

→

3 · Preview
Row by row: new / update / duplicate / error / missing info, with the reason.

→

4 · Commit
Chunked transactional writes. Every row tagged with `importJobId`.

→

5 · Report / Undo
Excel report of what happened. Undo deletes what the job wrote (never books that were since ordered) and restores stock.

`src/modules/admin/data-import/values.ts` — reading a hand-typed amount

```ts
export const INVALID = Symbol('invalid');
export type Parsed<T> = T | null | typeof INVALID;
```

Every parser answers one of three things: a clean value, null (the cell is empty, that's fine), or INVALID (something is written but unreadable). That third answer lets the preview point at the exact row and cell to fix, instead of silently guessing.

```ts
export function latinDigits(s: string): string {
  return s
    .replace(/[٠-٩]/g, (d) => String(d.charCodeAt(0) - 0x0660))
    .replace(/[۰-۹]/g, (d) => String(d.charCodeAt(0) - 0x06f0))
    .replace(/٫/g, '.') // ٫ decimal separator
    .replace(/٬/g, ','); // ٬ thousands separator
```

"١٥٠٠" becomes "1500". Each Arabic-Indic digit is a fixed distance from "0" in Unicode, so subtracting 0x0660 gives the digit.

```ts
}

export function parseAmount(raw: unknown): Parsed<number> {
  if (typeof raw === 'number') return Number.isFinite(raw) ? raw : INVALID;
  let s = latinDigits(text(raw));
  if (!s || /^[—–-]+$/.test(s)) return null;
```

A real number cell is used as is. A dash ("—") means "nothing" and counts as empty.

```ts
  s = s
    .toLowerCase()
    .replace(/(دج|د\.ج\.|\s?ج|دج|دينار|dzd|da|dinars?|€|eur|\$|usd)/g, '')
    .replace(/[\s ']/g, '');
```

Strip currency words in Arabic, French and English ("دج", "DA", "DZD", "dinars", "€") and spaces: «1 500 دج» → "1500".

```ts
  if (!/^-?[\d.,]+$/.test(s)) return INVALID;

  const lastDot = s.lastIndexOf('.');
  const lastComma = s.lastIndexOf(',');
  if (lastDot !== -1 && lastComma !== -1) {
    // Both: whichever comes last is the decimal separator.
    s = lastComma > lastDot ? s.replace(/\./g, '').replace(',', '.') : s.replace(/,/g, '');
```

▸ "1.500,50" (French) and "1,500.50" (English) both become 1500.5.

```
  } else if (lastComma !== -1) {
    s = /^-?\d{1,3}(,\d{3})+$/.test(s) ? s.replace(/,/g, '') : s.replace(',', '.');
```

▸ "1,500" = thousands → 1500, but "12,5" = decimal → 12.5.

```
  } else if (lastDot !== -1) {
    // "1.500" / "1.500.000" is how prices are written in Algeria (French usage).
    if (/^-?\d{1,3}(\.\d{3})+$/.test(s)) s = s.replace(/\./g, '');
```

▸ A local detail that matters: in Algeria "1.500" means one thousand five hundred dinars, not one and a half.

```
  }
  const n = Number(s);
  return Number.isFinite(n) ? n : INVALID;
}
```

Also handled: `parseYear` turns "1420هـ" (or a bare 13xx/14xx year) into a note instead of a wrong Gregorian year. `parseNames` splits "ابن تيمية، ابن القيم" into two authors. Dates, payment methods and expense categories are recognised in three languages. Duplicates are found in memory on title + main author, and author/publisher names are resolved in bulk, which is why 5,000 rows import in seconds.

PART II · CHAPTER 15

Cover pictures

A phone photo is about 1 MB and 3000+ px wide, but a book card shows it about 200 px wide. Each cover is processed once, at upload, so every visitor gets a light, fast image.

src/common/utils/cover-image.ts — processCover()

```ts
const FULL_MAX_PX = 2000;
const THUMB_WIDTH = 480;
/** Refuse decompression bombs (a tiny file that expands to gigapixels). */
const MAX_INPUT_PIXELS = 60_000_000;
```

Three dials: the big picture is capped at 2000 px (still sharp full-screen), cards use a 480 px thumbnail, and any image over 60 megapixels is refused. A 50 KB file can claim to be 100,000 × 100,000 px and would crash the server's memory.

```ts
export async function processCover(input: Buffer): Promise<ProcessedCover> {
  const base = sharp(input, { limitInputPixels: MAX_INPUT_PIXELS, failOn: 'error' }).rotate();
```

.rotate() with no angle reads the phone's EXIF "orientation" tag and turns the pixels upright, so sideways photos are fixed. sharp drops all metadata on output by default, including the GPS location phones embed (your home or shop address).

```ts
  const { data: full, info } = await base
    .clone()
    .resize({ width: FULL_MAX_PX, height: FULL_MAX_PX, fit: 'inside', withoutEnlargement: true })
    .jpeg({ quality: 85, mozjpeg: true })
    .toBuffer({ resolveWithObject: true });
```

The full version: fit inside 2000 × 2000, never enlarge a small picture, encoded with mozjpeg (about 20–30 % smaller than a standard JPEG at the same quality).

```ts
  const thumb = await base
    .clone()
    .resize({ width: THUMB_WIDTH, withoutEnlargement: true })
    .jpeg({ quality: 74, mozjpeg: true })
    .toBuffer();
```

The thumbnail, stored next to it in R2 as "<name>.thumb.jpg". The API returns its address as thumbUrl. Clients show it and fall back to the full picture if it's missing.

```ts
  return { full, thumb, width: info.width, height: info.height };
}
```

Where pictures come from

- Admin upload / mobile Quick-Add camera → `POST admin/uploads/cover`
- The real file type is decided by its first bytes ("magic bytes"): JPG, PNG or WEBP only
- `image-sync/` matches a folder of phone photos to books by file name + OCR
- `images/fill` looks up missing covers online

Result

- Storefront pages load about **20× less image data**
- No GPS or private metadata ever published
- R2 has no egress fees, so serving covers stays nearly free
- `scripts/backfill-thumbnails.ts` created thumbnails for older covers (safe to re-run)

PART II · CHAPTER 16

Packs (plans)

The platform is sold in three packs. One decorator locks a feature on the server, and one component locks it in the admin.

src/common/plan/plans.ts — what each pack unlocks

```ts
export const FEATURES = ['money', 'invoices', 'reports', 'socialExport', 'historyImport', 'academic']
as const;
```

"as const" turns the list into a TypeScript type. Misspelling a feature name anywhere is a compile error, not a silent bug.

```ts
export const PLANS: Record<PlanKey, PlanDef> = {
  starter: {
    label: 'أساسي',
    features: [],
    limits: { books: 1000, importsPerMonth: 1 },
  },
  pro: {
    label: 'مكتبة',
    features: ['money', 'invoices', 'reports', 'socialExport', 'academic'],
    limits: { books: 10_000, importsPerMonth: null },
  },
  business: {
    label: 'مؤسسة',
    features: [...FEATURES],
    limits: { books: null, importsPerMonth: null },
  },
};
```

The whole commercial offer in one object. null = unlimited. Core selling (storefront, orders, books, stock, apps, catalogue export) is in EVERY pack, on purpose: a shop must always be able to leave with its data.

```ts
export function cheapestWith(feature: Feature): PlanKey {
  return PLAN_KEYS.find((k) => PLANS[k].features.includes(feature)) ?? 'business';
}
```

For the upgrade message: "available from the «مكتبة» pack".

src/common/plan/requires-feature.ts — locking a route

```ts
async canActivate(ctx: ExecutionContext): Promise<boolean> {
  const feature = this.reflector.getAllAndOverride<Feature | undefined>(FEATURE_KEY,
[ctx.getHandler(), ctx.getClass()]);
  if (!feature) return true;
```

Read the @RequiresFeature('reports') label from the route (or its controller). No label → nothing to check.

```ts
  const plan = planOf(await this.stores.getPlan());
  if (plan.features.includes(feature)) return true;
  const needed = cheapestWith(feature);
  throw new PlanRequiredException(
    `«${FEATURE_LABELS[feature]}» غير متوفرة في باقة «${plan.label}». متوفرة ابتداءً من باقة
«${PLANS[needed].label}».`,
    { feature, plan: plan.key, requiredPlan: needed },
  );
```

Not in the pack → HTTP 402 "Payment Required", with an Arabic message the admin shows as-is, plus machine-readable fields for the apps.

```
}

export function RequiresFeature(feature: Feature) {
  return applyDecorators(SetMetadata(FEATURE_KEY, feature), UseGuards(FeatureGuard));
}
```

Usage on a controller: @RequiresFeature('money') ABOVE @UseGuards(AdminAuthGuard). Decorators apply bottom-up, so authentication runs first and a stranger gets 401, never a hint about your pack.

admin/src/components/admin/plan-gate.tsx — the same lock in the interface

```
export function useFeature(feature: Feature): boolean {
  const { data } = usePlan();
  return data ? data.features[feature] : true;
}
```

Returns "allowed" while the pack is still loading, so menu items don't flash a lock on first paint.

```
export function FeatureGate({ feature, children }: { feature: Feature; children: ReactNode }) {
  const { data, isLoading } = usePlan();
  if (isLoading || !data) return <TableSkeleton rows={4} cols={3} />;
  if (data.features[feature]) return <>{children}</>;
  return <UpgradeCard feature={feature} plan={data} />;
}
```

Wrap a page in <FeatureGate feature="reports">...</FeatureGate>. Inside the pack you see the page; outside it you see a friendly card explaining what the feature does (in Arabic) and which pack includes it. The server check above is the real lock; this one is for a good experience.

PART II · CHAPTER 17

Arabic search

Customers type احمد for أحمد, and مكتبه for مكتبة. Search has to forgive those variants.

src/common/utils/normalize-arabic.ts — normalizeArabic()

```ts
export function normalizeArabic(text: string): string {
  return text
    .toLowerCase()
```

Latin letters in titles ("SILA", "Python") match whatever case is typed.

```ts
    .replace(/[ً-ٟ]/g, '')
```

Remove the tashkeel (fatha, damma, kasra, shadda, sukun, tanween…) and the dagger alif. "الْكِتَاب" → "الكتاب".

```ts
    .replace(/[أإآا]/g, 'ا')
```

All forms of alif with hamza or madda → plain alif: أحمد = احمد.

```ts
    .replace(/ة/g, 'ه')
```

Taa marbuta ↔ haa: مكتبة = مكتبه.

```ts
    .replace(/ى/g, 'ي')
```

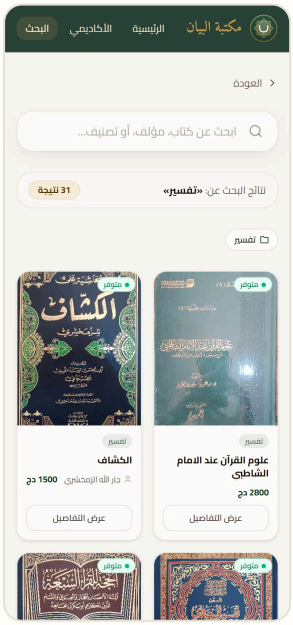
Alif maqsura ↔ yaa: مصطفى = مصطفي.

```ts
    .replace(/\s+/g, ' ')
    .trim();
```

Collapse repeated spaces.

```ts
}
```

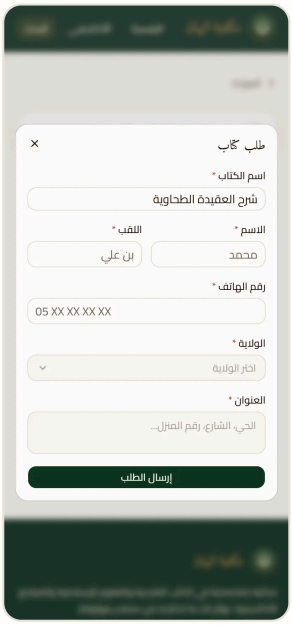
The same function runs on both sides. When a book is saved, its normalised title goes into `Book.titleNormalized`. When a customer searches, the query is normalised the same way, then matched with `contains` (case-insensitive) against that column, along with authors and publishers. So both sides are always compared in the same "flattened" spelling. `scripts/perf/explain.ts` shows the database plan for this query.



Search on a phone



Order form

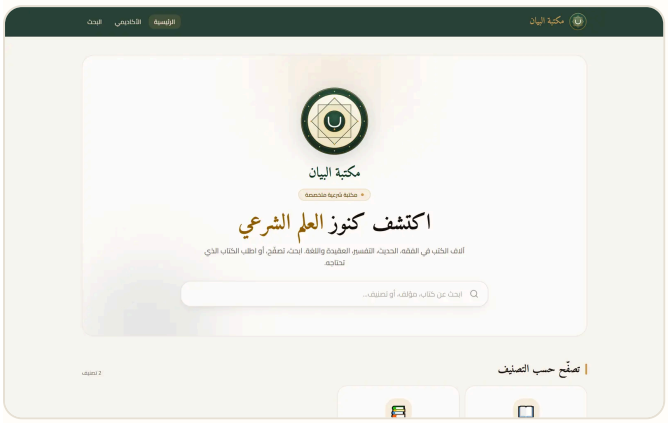


"Request a book"

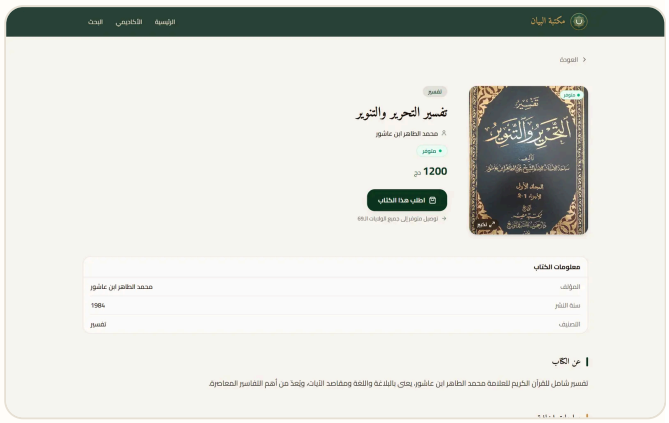
PART III · CHAPTER 18

The storefront

`frontend/`: what customers see. It's built for speed on a mid-range phone with a weak 4G signal.



Home: search hero + blocks in the order the admin set



Book page: gallery, details, order / ask on WhatsApp

Page	What it does
`home.tsx`	Fixed search hero, then the blocks from `GET /home-sections`: categories, latest, popular, request form, and hand-picked shelves.
`search.tsx`	Instant results by title, author and publisher, with Arabic normalisation.
`book.tsx`	Full book: gallery, description, extra info, stock status, order and request dialogs.
`collection.tsx`	A shelf's «عرض الكل» (see all) page.
`academic/*`	Field → year → subject → books (full edition only).

Why it's fast

- **Slim list responses:** lists return only what a card needs (`bookCardInclude`). Home asks for 16 books, not the whole catalogue.
- **Thumbnails** on cards (chapter 15).
- **Hashed files cached for a year:** the browser re-downloads only what changed after a deploy.
- **Heavy code loads on demand**, as shown below.

`frontend/src/components/lazy-dialogs.tsx` — loading the order dialog only when needed

```
const loadRequest = () => import("@/components/request-dialog");
const loadOrder = () => import("@/components/order-modal");
const RequestImpl = lazy(loadRequest);
const OrderImpl = lazy(loadOrder);
```

▸ import() inside a function = "download this file later". Vite splits the dialogs (and their dialog library, the heaviest part of the storefront) into separate small files that the first page load skips.

```
function useLazyTrigger(trigger: ReactElement, preload: () => Promise<unknown>) {
  const [active, setActive] = useState(false);
  const open = () => setActive(true);
  const warm = () => void preload().catch(() => undefined);
```

```
  const element = cloneElement(trigger, {
    role: "button",
    tabIndex: 0,
    "aria-haspopup": "dialog",
    onClick: open,
    onPointerEnter: warm,
    onTouchStart: warm,
    onFocus: warm,
```

▸ The trick: the code starts downloading the moment a finger TOUCHES the button (or the mouse hovers it), a fraction of a second before the click. By the time the tap completes, the dialog is usually ready, so it feels instant without slowing the first page load. role/tabIndex/aria keep it usable with a keyboard and screen readers.

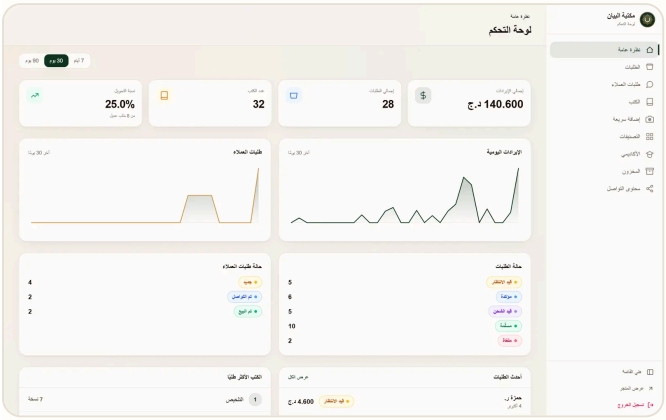
```
    // …
  });
}
```

Rule: always import the order and request dialogs from `components/lazy-dialogs.tsx`, never directly. A direct import would pull the heavy code back into the first page load.

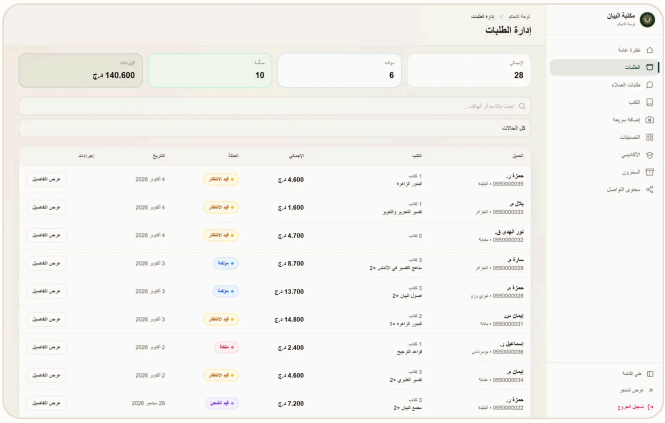
PART III · CHAPTER 19

The admin dashboard

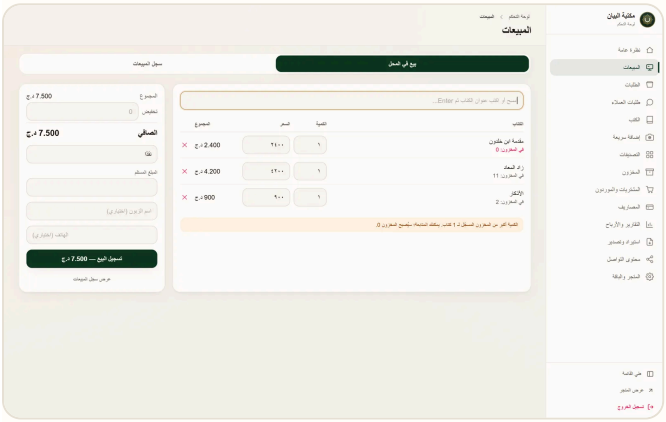
`admin/`: a separate Vite app served under `/admin/`. It's deployed on its own, so customers never download back-office code.



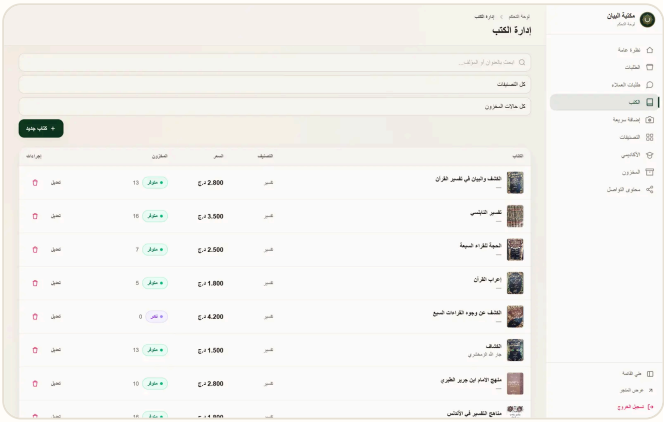
Overview: today's numbers and what needs attention



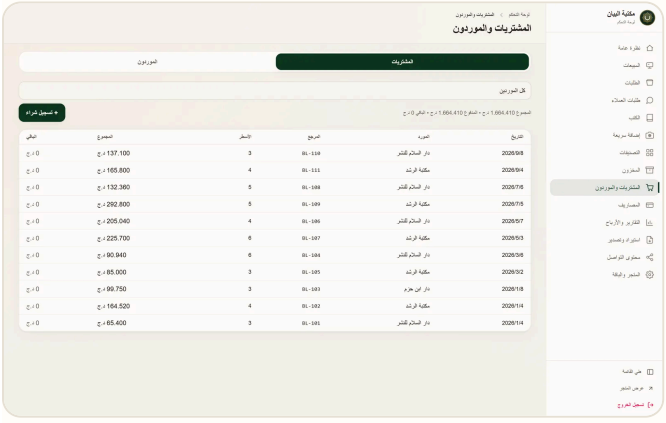
Orders: one register for every channel



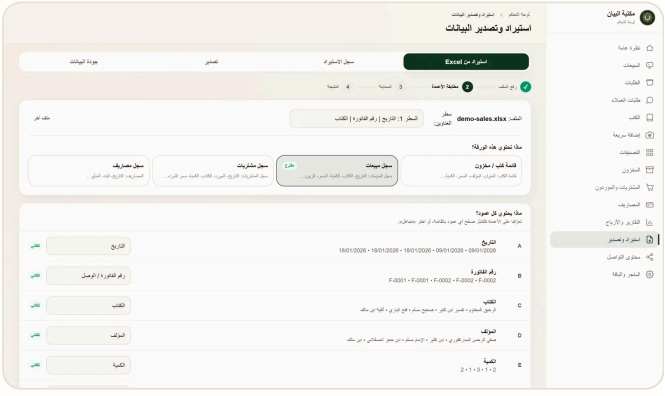
Counter sales (POS)



Books, with "missing price / cover / stock" filters



Purchases from suppliers



Excel wizard: choosing what the sheet contains

Every admin screen

Screen	Purpose	Pack
overview	Key numbers, recent orders, alerts	all
orders / order-detail	Status workflow, edit lines, discount, print invoice, return	all
requests	Customer leads → convert to an order, or delete spam	all
sales	Counter POS + the register of all sales	all
books / quick-add	Catalogue CRUD, gallery; fast entry on a phone	all
catalog	Authors, publishers, categories (with book counts)	all
inventory	Stock counts, statuses, movement history per book	all
storefront	Arrange the home page: reorder, rename, hide blocks; build shelves	all
data	Import wizard, history + undo, exports, data-quality report	all history: Business
purchases / expenses	Suppliers, stock in, payments, balance; shop expenses	Pro+
invoice	Printable numbered invoice	Pro+
reports	Profit and loss, margins, top and slow books, Excel export	Pro+
social-content	Pick books → ZIP of covers + captions for Instagram/Facebook/TikTok	Pro+
academic	Fields, years, subjects (full edition)	Pro+
settings	Store identity for invoices; current pack	all

admin/src/lib/auth.ts — keeping the admin logged in

```
const TOKEN_KEY = "bayan-admin-token";
export const getToken = () ⇒ localStorage.getItem(TOKEN_KEY);
```

▸ After login, the 30-day JWT is kept in the browser's localStorage.

```
export function attachAuth(instance: AxiosInstance) {
  instance.interceptors.request.use((config) ⇒ {
    const token = getToken();
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });
```

▸ An "interceptor" runs on EVERY request the admin makes. No page has to remember to add the token; it is added here, once.

```
  instance.interceptors.response.use(
    (res) ⇒ res,
    (err) ⇒ {
      if (err?.response?.status ⩵ 401) redirectToLogin();
      return Promise.reject(err);
    },
  );
```

> And on every response: if the server says 401 (token expired or invalid), clear it and go to /admin/login. A full page reload also clears any cached data.

```
}
```

Every admin page is `React.lazy` in `App.tsx`, so opening "Orders" doesn't download the code of "Reports". Data comes through TanStack Query hooks, which cache results and refresh them in the background. After a change, the related queries are refreshed, so lists update without a reload.

PART III · CHAPTERS 20–21

The mobile apps, and editions

20. The mobile apps (Flutter)

They live in a separate repository (`BookStoreMobile`) and are the native twins of the two web apps. They use the same palette, fonts, logo and Arabic RTL layout, and they call this same API over HTTP. The backend doesn't care which app is calling.

store_app

Browse, search, book page, order and request. Uses `GET /books` without `limit` to load the catalogue.

admin_app

JWT login, the orders and requests workflow, and **Quick-Add**: camera → cover uploaded to R2 → title + category → "save & scan next".



Quick-Add on a phone

Porting rule: a UI change on the web isn't automatic on mobile; it must be ported by hand. Pending ports are listed in the roadmap.

21. Editions: one codebase, two products

The same code builds two products. **basic** is the build for مكتبة البيان. **full** is the complete platform you sell to other stores. Hidden features stay in the code and the database; only the menus and pages are switched off.

`frontend/src/lib/features.ts` — the edition switch

```
const EDITION = (import.meta.env.VITE_EDITION as string | undefined) ?? "basic";
```

▸ Vite replaces import.meta.env.VITE_EDITION with a fixed value when the app is BUILT (set in render.yaml). It is a build-time setting, not something a visitor can change.

```
export const features = {
```

```
  academic: EDITION === "full",
};
```

> Pages ask "features.academic ?" before showing the academic nav link and home block. /academic/* routes redirect home in basic. The admin has its own copy (admin/src/lib/features.ts), and the Flutter apps use --dart-define=EDITION=full.

Editions vs. packs: an *edition* is decided when the app is built: which product it is. A *pack* is decided per store at run time: what that customer paid for. Both exist so one codebase can be sold in different shapes.

PART IV · CHAPTER 22

Deployment and operations

`render.yaml` describes all three services. Pushing to `master` redeploys them.

Service	How it runs
bookstore-api	Docker (Node 22). Two-stage build: compile, then copy only runtime packages. Runs `prisma migrate deploy` on boot. Health check `/api/v1/health`.
bookstore-storefront	Static site from `frontend/dist`. Hashed assets cached for a year.
bookstore-admin	Static site from `admin/dist`, base `/admin/`.
Database	Neon PostgreSQL: a pooled URL for the app, a direct URL for migrations.
Secrets	Entered once in the Render dashboard (`sync: false`). `JWT_SECRET` is generated by Render and never shown.

src/common/keep-alive.ts — staying awake on the free plan

```ts
export function keepAwake(): void {
  const base = process.env.RENDER_EXTERNAL_URL?.replace(/\/$/, '');
  if (!base || process.env.KEEP_AWAKE === 'false') return;
```

Only runs on Render (which sets RENDER_EXTERNAL_URL). It does nothing on your Mac. Set KEEP_AWAKE=false once you move to a paid plan.

```ts
  const url = `${base}/api/v1/health`;
  const log = new Logger('KeepAwake');
  const ping = () =>
    fetch(url, { signal: AbortSignal.timeout(30_000) }).catch((e: Error) => log.warn(`ping failed: ${e.message}`));
  setInterval(ping, 10 * 60_000).unref();
```

Render's free plan sleeps after 15 minutes without traffic, and the next customer then waits about 30 s. Calling its own public URL every 10 minutes counts as traffic, so it never sleeps. One always-on service uses about 744 of the 750 free hours a month. .unref() means the timer never keeps the process alive during shutdown.

```ts
  log.log(`pinging ${url} every 10 min`);
}
```

Useful scripts

Script	Use
`scripts/setup-store.ts`	Set the store's name, slug and identity
`scripts/set-plan.ts`	Change a store's pack (starter / pro / business)
`scripts/backfill-thumbnails.ts`	Create missing 480 px thumbnails (safe to re-run)
`scripts/migrate-covers-to-r2.ts`	Move old local covers into R2
`scripts/perf/*`	Seed a throwaway N-book store, benchmark endpoints, explain search. Local only.

PART IV · CHAPTER 23

Testing and quality

The riskiest logic (money, stock statuses, parsing user files) is written as **pure functions**, with no database and no framework, so it can be tested in milliseconds.

Test file	What it proves
`orders/order-rules.spec.ts`	Allowed and refused status moves; totals, discount clamping, rounding
`reports/money-logic.spec.ts`	Margins, COGS, purchase balances
`data-import/data-import.spec.ts`	Header recognition, «1 500 ج.م», "1.500,00", Hijri years, name splitting
`utils/normalize-arabic.spec.ts`	Alif/hamza, taa marbuta, tashkeel
`utils/cover-image.spec.ts`	Resize, thumbnail naming, thumbUrl rules
`utils/export-safety.spec.ts`	Excel formula injection is neutralised

Status today: 6 suites and 151 tests, all passing. The TypeScript build is clean. `npm audit` reports 0 known vulnerabilities on the API after the November 2026 patch (body-parser, multer, qs).

Commands

terminal — day-to-day

```
npm run start:dev                  # API on :3000, restarts on save
cd frontend && npm run dev         # storefront on :5173
cd admin && npm run dev            # admin on :5175/admin/
npx jest                           # run all unit tests
npm run build                      # type-check + compile the API
npx prisma migrate dev --name x    # create a migration after editing schema.prisma
npm audit --omit=dev               # check runtime packages for known vulnerabilities
```

PART V · CHAPTER 24

What makes this platform good

These are the principles that show up again and again in the code, and the reason a small shop can trust it with its money.

1. One source of truth per fact

Stock changes only through `moveStock()`. Money is computed only in `computeOrderTotals()`. A sale is defined once (`SOLD`, `SALE_DATE`). Numbers can't drift apart between screens.

2. All-or-nothing writes

A sale, its stock move and its log are one transaction. A crash halfway leaves nothing half-done.

3. Safe under double clicks

Conditional updates (`where: { status: old }`, `invoiceNumber: null`) and atomic SQL mean two clicks or two cashiers never sell or number twice.

4. History is frozen

Orders copy the customer, prices and unit cost. Changing a price today never rewrites last year's invoices or margins.

5. Everything is traceable and undoable

Every stock change has a reason and a link. Every import is tagged and can be undone. A return is a normal, logged status change.

6. Built for the real local shop

Arabic-first RTL, Algerian number formats, 69 wilayas, WhatsApp ordering, legal IDs on invoices, Hijri years, French/Arabic Excel headers.

7. Fast on cheap phones and cheap servers

Thumbnails, slim list responses, one SQL query per read, caches, code split on demand, year-long asset caching, a free plan that never sleeps.

8. Secure by default

Fails closed, filters by store everywhere, hides cost prices, rate-limits, blocks SSRF and formula injection, strips GPS from photos.

9. Your data stays yours

Catalogue and orders export to Excel in every pack. Leaving the platform is never a trap, and that's a selling point.

10. One codebase, many products

Editions (build time) and packs (per store) let the same code serve مكتبة البيان and be sold to other shops.

11. Readable on purpose

The same module shape everywhere. Comments explain *why*, not *what*. Pure rule files with tests. Files are small and named for what they do.

12. Cheap to run

Render free tier, Neon, R2 with no egress fees. The whole platform can run for almost nothing until sales justify more.

PART V · CHAPTER 25

Roadmap

Where the platform has been, where it stands in November 2026, and the next steps in the order they make sense.

Foundation shipped

Catalogue, academic tree, storefront, requests via WhatsApp, admin dashboard, JWT auth, R2 covers, mobile apps, production hardening (rate limits, tenancy, helmet), social export.

Phase 1: Smart import and export shipped

Excel wizard (analyze → preview → commit → report → undo), exports, data-quality report.

Phase 2: Money shipped

Suppliers, purchases, payments, expenses, counter POS, one sales register, numbered invoices, returns.

Phase 3: Reports shipped

Profit and loss per month, COGS from frozen costs, margins, top and slow books, stock value, Excel export.

Phase 4: Packs shipped

Starter / Pro / Business, server and UI gating, limits. Then: admin-arranged home page and hand-picked shelves, deleting requests, and the security patch (Nov 2026).

Now: small housekeeping next

- Merge `master` into `develop` so both branches match again.
- Move the `shadcn` CLI to `devDependencies` in `admin/` and `frontend/` (clears most build-tool audit warnings).
- Commit the `facebook/` brand images; add `.DS_Store` to `.gitignore`.

Next: mobile catch-up planned

- `store_app`: render the hand-picked shelves from `/home-sections`.
- `admin_app`: money (purchases, expenses), reports, pack gating, deleting requests.

Then: ready for many stores planned

- **Per-user accounts** (owner, cashier…) instead of the shared `ADMIN_PASSWORD`, with roles and who-did-what in the logs.
- **Platform-owner console:** create stores, set packs, see usage, without running scripts.
- Then several stores per deployment, using the `storeId` already on every row.

Later: platform upkeep when quiet

- Prisma 5 → 7, NestJS 11 → 12, TypeScript 7: one planned upgrade week, with the test suite as the safety net.
- Paid Render plan once revenue allows (then set `KEEP_AWAKE=false`).
- Error tracking (the exception filter is already Sentry-ready).

CHAPTER 26

Glossary

Term	Meaning
API	The server program the apps talk to. It receives requests, applies rules, reads and writes the database, and answers in JSON.
Endpoint / route	One address of the API, e.g. `GET /api/v1/admin/orders`.
Module / controller / service	NestJS building blocks: a feature folder, its URL handlers, and its logic.
DTO	"Data transfer object": a class describing what a request body must look like. Used for validation.
Guard	Code that runs before a route and decides whether the request may continue (login, pack).
Decorator	The `@Something()` labels on classes and methods that attach behaviour (routes, guards, validation).
ORM / Prisma	The library that turns typed TypeScript calls into SQL and keeps the schema in versioned migrations.
Migration	A saved SQL change to the database structure, applied in order on every environment.
Transaction	A group of database writes that all succeed or are all cancelled together.
Atomic	Happens as one indivisible step; nobody can see or interfere with a half-done state.
JWT	"JSON Web Token": a signed pass the server gives at login and checks on every admin request.
CORS	Browser rule deciding which websites may call the API.
Rate limit	A maximum number of requests per minute per visitor.
SSRF	"Server-side request forgery": tricking a server into fetching an internal address for an attacker.
COGS	Cost of goods sold: what the sold books cost you to buy.
Tenancy / storeId	Keeping each store's data separate in a shared database.
Thumbnail	A small version of a picture for lists and cards.
Bundle / code splitting	The JavaScript files a browser downloads; splitting means downloading parts only when needed.
TanStack Query	Front-end library that fetches, caches and refreshes server data for the pages.
Edition / pack	Edition = which product the app was built as (basic/full). Pack = what a given store pays for.